Transformations and GLM 4

4.1 Transformations

Models are composed of vertices and the operations like moving, or changing the scale of the represented objects may be achieved by applying the same operation to everyone of these vertices. For instance we know that a 2D rotation can be expressed as

$$\underbrace{\begin{bmatrix} x' \\ y' \end{bmatrix}}_{\mathbf{p}'} = \underbrace{\begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix}}_{\mathbf{R}} \underbrace{\begin{bmatrix} x \\ y \end{bmatrix}}_{\mathbf{p}} = \begin{bmatrix} x\cos\theta - y\sin\theta \\ x\sin\theta + y\cos\theta \end{bmatrix}.$$

One of the advantages of this matricial representation based on matrix-vector multiplications is that a single operation may be used encode the application of the same transformation to a set of points. For the case of 2D the rotation on a set of N points it is sufficient to glue them together as columns of a matrix and apply the transformation as follows:

$$\begin{bmatrix} x'_0 & x'_1 & \dots & x'_N \\ y'_0 & y'_1 & \dots & y'_N \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x_0 & x_1 & \dots & x_N \\ y_0 & y_1 & \dots & y_N \end{bmatrix}$$

or

$$\begin{bmatrix} \mathbf{p}'_0 & \mathbf{p}'_1 & \dots & \mathbf{p}'_N \end{bmatrix}_{(2\times N)} = \mathbf{R}_{(2\times 2)} \begin{bmatrix} \mathbf{p}_0 & \mathbf{p}_1 & \dots & \mathbf{p}_N \end{bmatrix}_{(2\times N)}.$$

Unfortunately this is not the case of translations, as we will need to create a second matrix with the same number of columns by repeating the translation vector as

$$\begin{bmatrix} \mathbf{p}'_0 & \mathbf{p}'_1 & \dots & \mathbf{p}'_N \end{bmatrix}_{(2\times N)} = \begin{bmatrix} \mathbf{t} & \mathbf{t} & \dots & \mathbf{t} \end{bmatrix}_{(2\times N)} + \begin{bmatrix} \mathbf{p}_0 & \mathbf{p}_1 & \dots & \mathbf{p}_N \end{bmatrix}_{(2\times N)}.$$

4.2 Homogeneous transformations

It is common to embed a transformation in an nD space in a $(n+1)$D one to linearise or take advantage otherwise. This is what we can do with the concept of homogeneous coordinates. In fact it simply consists in adding an additional component to the coordinates of points or vectors. So a 2D point $[x, y]$ becomes $[x, y, 1]$ in homogeneous coordinates. Additionally any point $[\alpha x, \alpha y, \alpha]$ represents the same point as the homogenisation of the coordinates can be done always by multiplying the whole vector by the scale factor $1/\alpha$. This means that the additional coordinate can never be zero in this representation, although this restriction does not apply in *projective geometry* often used in computer vision.

Now we can describe a translation by a matrix multiplication as

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} x + t_x \\ y + t_y \\ 1 \end{bmatrix}$$

It is easy to understand that the rotation is

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

The composition of a rotation followed by a translation becomes

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix} = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & t_x \\ \sin(\theta) & \cos(\theta) & t_y \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

Or writing as the transformation as a block matrix,

$$\begin{bmatrix} x' \\ y' \\ 1 \end{bmatrix} = \begin{bmatrix} \mathbf{R}_{(2\times 2)} & \mathbf{t}_{(2\times 1)} \\ 0 \quad 0 & 1 \end{bmatrix} \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}$$

4.3 Using GLM

To support matrix creation and basic linear algebra operations we can use the **GLM** library in C++, **PyGLM** equivalent for Python, or the **glm-js** equivalent for JavaScript.

In fact this library also brings in some features that are available in the shading GLSL language, that will be discussed later. It supports the creation of matrices and vectors commonly needed for OpenGL programming, as well as some associated operations.

In C++ you may use it as:

— **C++** —

```
glm::mat4 myMatrix = glm::mat4(1.0);
glm::vec4 myVector = glm::vec4(1,2,3,4);
//  multiply vector by identity matrix
glm::vec4 transformedVector = myMatrix * myVector;
```

In JavaScript it becomes:

— **JavaScript** —

```
var MyMatrix = glm.mat4(1.0);
var myVactor = glm.vec4(1,2,3,4);
// I know the following seems quite strange but it works :)
var transformedVector = MyMatrix['*'](myVector);
```

The definition of quaternions and the equivalent rotation matrices can be done as:

— **JavaScript** —

```
// this produces the quaternion
var q = glm.angleAxis(glm.radians(45),glm.vec3(0, 0,1));
// a 4x4 3D rotation matrix from this quaternion
var rmat = glm.toMat4(q);

var myVactor = glm.vec4(1,2,3,4);
var transformedVector = MyMatrix['*'](myVector);
```

Important note: GLM, GLSL, and other computing languages such as Fortran, store matrices in column-wise order. This is the opposite of what happens in C or C++, where the sequence of elements in memory correspond to row-after-row. This may be somewhat confusing as the rows output by mat.toString() are indeed the columns. By consequence, when you specify the elements of a matrix on its declaration (constructor) you are supplying them in column order.

Exercises

Start with the example **Transformations** that you can find on your examples folder. You should try to understand the given code and try to modify it, to implement the following exercises. Note that as GLM-JS only provides support for 4D vectors and matrices, you should use them to compute the

rotations and translations in the plane z=0, and then convert the results to the 2D vertex structures to be drawn. Don't use any other library.

Exercise 4.1
Make a (very small) blue triangle follow a sinusoid from the left to the right and reapear on the left side.

Exercise 4.2
Make that triangle rotate while it moves along the sinusoid.

Exercise 4.3
Draw a small rectangle that rotates under the moving triangle.

Exercise 4.4
Create a functional clock with three handles: hours, minutes, and seconds. The handles must be drawn as rectangles of three different sizes of your choice. For that you should use the following code to obtain the time information:

— **JavaScript** —

```
1 var today = new Date();
2
3 var hours = today.getHours();
4 var minutes =  today.getMinutes();
5 var secs =  today.getSeconds();
```